Atelier 5 - Conteneuriser et déployer LogiAI avec Docker

LogiAI reste l'exemple. À la séance précédente, vous avez testé l'API, protégé /v1 par une clé et fait appeler l'API par une page web. Aujourd'hui, vous la rendez transportable : l'API et la page tournent chacune dans un conteneur, et une seule commande démarre l'ensemble.

Ce que vous produisez : Atelier 5 avec son COMPTE_RENDU.md rempli, rendu par une pull request et tagué atelier5.

Étape	Ce que vous faites
1. Préparer	branche, Docker démarré, premier constat : ça ne marche que chez vous
2. Cours court	image, conteneur, Dockerfile, ports, réseau, volumes, Compose
3. Activité 1	l'API seule dans un conteneur ; trois expériences pour la casser
4. Cours court	Nginx, reverse proxy et origine unique
5. TP, sprint 1	la page web dans un conteneur Nginx qui relaie /api/
6. TP, sprint 2	Docker Compose, healthcheck, volumes, test de bout en bout, logs
7. Rendre	ADR 004, compte rendu, pull request, tag tp5, ticket de sortie

1 · Préparer (15 min)

Placez-vous à la racine de votre projet de l'Atelier 4, le dossier qui contient logiai/, web/, data/, models/ et .env. Adaptez les noms de dossiers à votre dépôt. Puis vérifiez :

```
docker --version
docker info
python -m pytest
```

docker info doit répondre sans erreur : Docker Desktop doit être démarré, avec le moteur Linux (WSL2). Si vous lisez « Cannot connect to the Docker daemon », le client fonctionne mais le moteur est arrêté. pytest doit être tout vert : l'Atelier 4 est la base de celui-ci.

- ✓ Vérifiez aussi que le modèle existe : models/model.joblib et models/model_info.json.
- ✓ Sinon : python -m logiai.dataset puis python -m logiai.train.
- ✓ Une image se construit avec ce qui est sur le disque au moment du build : sans modèle, pas de modèle dans l'image.

Premier constat : ça ne marche que chez vous

Listez ce que LogiAI suppose de votre machine, puis qui le garantit aujourd'hui.

Ce que le projet suppose	Qui le garantit aujourd'hui
Une version de Python et des bibliothèques compatibles avec model.joblib	vous, avec .venv et requirements.txt
Le fichier .env avec les deux clés	vous
Un modèle entraîné dans models/	vous, avec python -m logiai.train
Deux terminaux : uvicorn, puis python -m http.server 5500 pour la page	vous

À montrer : votre liste, et la phrase « si je donne mon dossier à un camarade, il doit… ».

2 · Docker en dix idées

Question	Réponse dans LogiAI
Qu'est-ce qui est figé ?	une image : Python, bibliothèques, code, données et modèle, en couches en lecture seule
Qu'est-ce qui tourne ?	un conteneur : un processus isolé, créé à partir d'une image
Comment décrire l'image ?	un Dockerfile : une instruction, une couche
Que voit le build ?	le contexte : le dossier courant, filtré par .dockerignore
Comment atteindre l'API ?	publier un port : hôte:conteneur
Comment deux conteneurs se parlent-ils ?	un réseau Docker, avec un DNS interne qui résout les noms de services
Que devient une donnée écrite dans le conteneur ?	elle disparaît avec lui, sauf si elle est dans un volume
Comment démarrer plusieurs services ?	Docker Compose : un fichier YAML, une commande
Comment savoir si l'API est prête ?	un healthcheck : un test exécuté dans le conteneur
Où sont les secrets ?	dans .env, transmis à l'exécution, jamais copié dans l'image

❖ Image et conteneur

Une image est un modèle immuable, empilement de couches en lecture seule. Un conteneur est une instance en exécution de cette image, avec une fine couche inscriptible éphémère. Avec une seule image, on lance plusieurs conteneurs indépendants. Supprimer un conteneur ne supprime pas l'image.

Analogie LogiAI : l'image est la fiche de chargement d'un camion (quoi mettre, dans quel ordre) ; le conteneur est un camion chargé selon cette fiche, qui roule. On peut charger dix camions avec la même fiche. Un conteneur n'est pas une machine virtuelle : il partage le noyau de l'hôte, d'où sa légèreté. Sous Windows, Docker Desktop fournit ce noyau Linux avec WSL2.

❖ Le Dockerfile, couche par couche

Instruction	Rôle	Piège
FROM	image de base (python:3.11-slim : Python, sans le superflu)	tag précis, pas de latest
WORKDIR	répertoire de travail dans l'image	créé s'il n'existe pas
COPY	copie du contexte vers l'image	ne voit que ce qui est dans le contexte
RUN	commande exécutée pendant la construction	une couche de plus à chaque RUN
ENV	variable disponible dans le conteneur	pas de secret ici
EXPOSE	documente le port d'écoute	ne publie rien
CMD	commande par défaut au démarrage	remplaçable en fin de docker run

Le cache. Docker réutilise une couche si l'instruction et les fichiers qu'elle copie n'ont pas changé. Dès qu'une couche change, toutes les suivantes sont refaites. On place donc le plus stable en premier : les dépendances avant le code, le code avant les données et le modèle, qui changent à chaque entraînement.

Le contexte et .dockerignore. Le point de docker build -t nom . est le contexte : Docker envoie ce dossier au moteur. Sans filtre, il enverrait .venv, .git et surtout .env, qui contient vos clés. .dockerignore exclut ces fichiers : la clé ne peut pas finir dans l'image.

❖ Ports, réseau et trois sens de localhost

8001:8000 se lit hôte:conteneur. L'API écoute sur le port 8000 dans le conteneur, et Windows l'atteint sur le 8001. On choisit 8001 pour ne pas entrer en conflit avec un uvicorn de l'Atelier 4 qui occuperait encore le 8000.

Où êtes-vous ?	localhost désigne
Dans votre navigateur, sur Windows	votre machine

Où êtes-vous ?	localhost désigne
Dans le conteneur api	ce conteneur lui-même
Dans le conteneur web	ce conteneur lui-même : l'API se joint par son nom de service, api

Pour que l'API soit joignable depuis l'extérieur de son conteneur, uvicorn doit écouter sur 0.0.0.0. Avec 127.0.0.1, seul l'intérieur du conteneur pourrait lui parler, et le port publié ne servirait à rien.

❖ Volumes et Compose

COPY fige un fichier dans l'image au moment du build. Un bind mount (./models:/app/models) relie un dossier de votre machine au conteneur à l'exécution et masque le contenu du même dossier dans l'image. Conséquence : un nouveau model.joblib sur votre disque est vu par le conteneur sans reconstruire l'image.

Compose décrit les services, leur réseau, leurs ports, leurs volumes et leurs dépendances dans docker-compose.yml. **depends_on seul garantit l'ordre de démarrage, pas que l'API soit prête.** Avec condition: service_healthy, Compose attend que le healthcheck de l'API réussisse avant de démarrer la page.

Docker n'entraîne pas le modèle : il l'empaquette et l'exécute. L'entraînement reste python -m logiai.train.

3 · Activité 1 · L'API dans un conteneur (40 min)

.dockerignore

À la racine du projet, créez .dockerignore :

```
.venv/
.venv_test/
venv/
.git/
.gitignore
__pycache__/
*.py[cod]
.pytest_cache/
.ruff_cache/
build/
dist/
*.egg-info/
.env
.vscode/
.idea/
docs/
tests/
```

data/ et models/ ne sont pas exclus : l'API en a besoin. .env est exclu : il sera transmis à l'exécution, pas copié dans l'image. Et .env.example, qui ne contient que de fausses valeurs, reste sans risque.

Dockerfile de l'API

Votre projet de l'Atelier 4 liste ses dépendances dans requirements.txt et place le package dans logiai/. Créez Dockerfile :

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY logiai/ ./logiai/
COPY data/ ./data/
COPY models/ ./models/
ENV LOGIAI_HOME=/app
EXPOSE 8000
CMD ["uvicorn", "logiai.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Si votre dépôt a un pyproject.toml et un dossier src/, remplacez les trois lignes concernées par COPY pyproject.toml README.md ./, COPY src/ ./src/ et RUN pip install --no-cache-dir . ; le reste est identique. Vérifiez dans logiai/config.py comment la racine du projet est déterminée : LOGIAI_HOME n'a de sens que si le code la lit.

Pourquoi cet ordre ? requirements.txt puis pip install d'abord : cette couche, la plus lente, est réutilisée tant que les dépendances ne changent pas. Le code vient ensuite, puis les données et le modèle.

Construire, lancer, vérifier

```
docker build -t logiai:1.0 .
docker images
docker run -d --name logiai-api-test -p 8001:8000 --env-file .env logiai:1.0
docker ps
curl.exe http://localhost:8001/health
```

Trois minutes environ au premier build. -d : en arrière-plan ; -p : port hôte:conteneur ; --env-file : les variables de .env, sans que le fichier entre dans l'image. /health répond, et /docs affiche Swagger sur http://localhost:8001/docs.

Puis rappelez-vous l'Atelier 4 : la clé protège /v1. Testez avec le fichier livraison.json de la séance précédente :

```
curl.exe -i -X POST http://localhost:8001/v1/predict -H "Content-Type:
application/json" -d "@livraison.json"
curl.exe -i -X POST http://localhost:8001/v1/predict -H "Content-Type:
application/json" -H "X-API-Key: votre-cle" -d "@livraison.json"
```

Attendu : 401 sans clé, 200 avec la clé de votre .env. Le comportement de l'API n'a pas changé : seul l'endroit où elle tourne a changé.

À montrer : docker images, docker ps, un 401 et un 200 sur le port 8001.

Trois expériences pour casser le conteneur

Pour chacune, notez dans le compte rendu : prévu, obtenu, explication. Après chaque expérience : docker stop logiai-api-test, docker rm logiai-api-test.

1. **Sans --env-file.** Relancez le conteneur sans cette option. Que dit /health, et quelle réponse donne /v1/predict avec votre clé ? Pourquoi, alors que .env est dans votre dossier ?
2. **Sans publier le port.** Relancez sans -p 8001:8000. Le conteneur est Up, EXPOSE est dans le Dockerfile : pourquoi curl échoue-t-il ?
3. **Écoute sur 127.0.0.1.** Dans une copie du Dockerfile, remplacez --host 0.0.0.0 par --host 127.0.0.1, reconstruisez sous un autre tag (logiai:test) et relancez avec -p. Que se passe-t-il, alors que le port est publié ?

À montrer : les trois résultats observés, et l'explication de chacun en une phrase.

4 · Cours court : Nginx devant l'API

À la séance précédente, la page servie sur le port 5500 appelait l'API sur le port 8000 : deux origines, donc CORS. Ici, un seul point d'entrée : Nginx sert la page et relaie les appels /api/ à l'API.

Question	Réponse dans LogiAI
Qui sert la page ?	Nginx, comme fichier statique
Qui calcule la prédiction ?	FastAPI : Nginx ne prédit rien
Pourquoi un chemin relatif ?	le navigateur appelle le même hôte et le même port que la page
Pourquoi pas api:8000 dans la page ?	le nom api n'existe que dans le réseau Docker, pas sur Windows
Et CORS ?	page et API sont sur la même origine : pas de requête préliminaire à autoriser
Et la clé ?	inchangée : le navigateur envoie X-API-Key, Nginx la transmet

Analogie LogiAI : la réception de l'entrepôt. Le visiteur ne connaît que l'accueil (Nginx). Il ne sait pas dans quel bâtiment est le quai de chargement (l'API) : l'accueil transmet.

Le reverse proxy n'est pas une protection. La clé reste indispensable : Nginx relaie, il ne vérifie rien. Et la clé tapée dans la page reste visible dans l'onglet Network de F12, comme à l'Atelier 4.

TP · Page web, Nginx, Compose

Contexte. L'application doit démarrer avec une commande, depuis un dossier vide de tout environnement Python : une page sur http://localhost:8080, une API sur http://localhost:8001, et l'API déclarée prête avant que la page ne démarre.

Ce qui est fourni, ce que vous écrivez

Fichier	Rôle	À écrire ?
.dockerignore, Dockerfile	l'image de l'API	fait à l'activité 1
web/index.html	la page de l'Atelier 4	à modifier : une ligne
nginx/default.conf	serveur et reverse proxy	à écrire, deux TODO
web/Dockerfile	l'image de la page	à écrire
docker-compose.yml	les deux services	à écrire, cinq TODO
docs/adr/004-docker.md	deux décisions à justifier	à écrire
COMPTE_RENDU.md	à remplir au fil du TP	à remplir

Sprint 1 · La page dans un conteneur Nginx (40 min)

1. web/index.html : l'adresse de l'API devient un chemin relatif. Remplacez la valeur actuelle de API_URL par :

```
const API_URL = '/api/v1/predict';
```

2. Créez nginx/default.conf à partir de ce squelette :

```
server {
    listen 80;
    server_name localhost;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass ???;                  # TODO 1 : où va la requête ?
        proxy_set_header Host $host;
        proxy_set_header ???;            # TODO 2 : l'adresse du client
    }
}
```

3. Créez web/Dockerfile :

```
FROM nginx:alpine
COPY ??? /usr/share/nginx/html/index.html
COPY ??? /etc/nginx/conf.d/default.conf
EXPOSE 80
```

À montrer : les trois fichiers, et le chemin qu'emprunte /api/v1/predict, de la page à FastAPI, expliqué à voix haute.

Pistes, sans la solution

À écrire	Pistes
proxy_pass	le service s'appelle api et écoute sur 8000 ; un slash final retire le préfixe /api/ : /api/v1/predict arrive à FastAPI comme /v1/predict
en-tête client	X-Real-IP reçoit $remote_addr ; X-Forwarded-For reçoit $proxy_add_x_forwarded_for
COPY de la page	le contexte de build sera la racine du projet : les chemins commencent par web/ et nginx/
502 plus tard	api n'est pas joignable : service arrêté, mauvais nom, ou uvicorn sur 127.0.0.1

Sprint 2 · Docker Compose (40 min)

1. Arrêtez et supprimez le conteneur de test s'il existe encore : docker rm -f logiai-api-test.
2. Créez docker-compose.yml à partir de ce squelette :

```
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: logiai-api
    ports:
      - ???                  # TODO 1 : hôte:conteneur
    env_file:
      - ???                  # TODO 2 : les clés, hors de l'image
    environment:
      LOGIAI_HOME: /app
    volumes:
      - ./models:/app/models # TODO 3 : et data ?
    healthcheck:
      test: ???              # TODO 4 : interroger /health depuis le conteneur
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s

  web:
    build:
      context: .
      dockerfile: web/Dockerfile
    container_name: logiai-web
    ports:
      - '8080:80'
    depends_on: ???          # TODO 5 : attendre que l'API soit saine
```

3. Validez, démarrez, vérifiez :

```
docker compose config
docker compose up -d --build
docker compose ps
```

Attendu : logiai-api en Up (healthy) après quelques secondes, d'abord (health: starting) ; logiai-web en Up. Si l'API devient unhealthy : docker compose logs api.

4. Test de bout en bout. Ouvrez http://localhost:8080, saisissez votre clé, envoyez une livraison normale : la page affiche une probabilité avec alerte. Puis http://localhost:8001/docs et http://localhost:8001/health.
5. Même appel, par Nginx, avec curl.exe. Notez les codes pour quatre requêtes : /api/health ; /api/v1/predict sans clé ; avec une fausse clé ; avec la bonne clé.
6. Ouvrez F12, onglet Network, pendant un appel réussi. Vers quelle adresse part la requête ? Voyez-vous une requête OPTIONS préliminaire, comme à l'Atelier 4 sur le port 5500 ? Pourquoi ?
7. **Observer.** docker compose logs --tail=30 api, puis docker compose logs -f api pendant une prédiction (Ctrl+C pour quitter), puis docker stats : relevez CPU, mémoire, réseau et nombre de processus de logiai-api.
8. **Casser puis réparer.** docker compose stop api, puis envoyez une livraison depuis la page. Que reçoit-elle, et qu'affiche-t-elle ? Relancez : docker compose start api.
9. **Voir le volume.** docker compose exec api ls /app/models : ces fichiers sont ceux de votre disque. Pourquoi un nouveau model.joblib entraîné sur Windows serait-il vu sans reconstruire l'image ? Et un changement dans logiai/ ?

À montrer : docker compose ps avec l'API healthy, la page qui prédit sur le port 8080, les quatre codes de l'étape 5 et le résultat de l'étape 8.

Pistes, sans la solution

À écrire	Pistes
ports de l'API	8001 côté Windows, 8000 dans le conteneur ; les guillemets évitent que YAML lise le deux-points comme autre chose
env_file	le fichier .env à la racine ; il n'est pas dans l'image, mais Compose le lit au démarrage
healthcheck	l'image slim n'a pas curl : python -c avec urllib.request.urlopen sur http://127.0.0.1:8000/health, en forme de liste : CMD, python, -c, puis la chaîne
/health sans clé	/health reste public, sinon le healthcheck recevrait un 401 et l'API ne serait jamais healthy
depends_on	api avec condition: service_healthy, sur deux niveaux d'indentation
volumes	un montage par dossier : ./models et ./data, chacun vers /app/...

Décision : l'ADR 004

Dans docs/adr/004-docker.md :

- le modèle est-il copié dans l'image (COPY) ou monté en volume ?
- quels ports sont publiés : 8080 seulement, ou aussi 8001 ?

Pour chaque décision, écrivez ce que l'autre option ferait perdre. Aide : que se passe-t-il quand on ré-entraîne ? Que peut atteindre un visiteur qui contourne Nginx ?

Le compte rendu : COMPTE_RENDU.md

Remplissez-le au fil de la séance : votre liste du premier constat, vos deux Dockerfile et votre docker-compose.yml, les trois expériences de l'activité 1 (prévu, obtenu), les codes de l'étape 5, ce que la page affiche à l'étape 8, vos relevés de docker stats, cinq questions de compréhension et le lien de la pull request.

Rendre le TP (15 min)

```
docker compose config                # valide
git status                           # .env ne doit PAS apparaître
git add .
git commit -m "Atelier 5 : Docker, Nginx et Compose"
git push
# GitHub : Compare & pull request ; un coéquipier relit, commente, approuve
# Merge pull request (Create a merge commit)
git switch main
git pull
git tag tp5
git push --tags
```

Le relecteur vérifie : docker compose up -d --build fonctionne chez lui, avec son propre .env ; aucun fichier .env dans les changements ; l'ADR 004 et COMPTE_RENDU.md sont remplis ; au moins un commentaire. Ne mettez jamais vos clés dans une capture d'écran : masquez-les.

Arrêter proprement

Action	Commande
Arrêter sans supprimer	docker compose stop
Relancer	docker compose start
Supprimer conteneurs et réseau	docker compose down
Reconstruire après modification du code	docker compose up -d --build

docker compose down ne supprime ni les images, ni vos dossiers data/ et models/. N'ajoutez pas -v sans savoir ce que cette option supprime.

Critères de réussite

- l'image de l'API se construit sans erreur ;
- .env n'est ni dans l'image ni dans Git ;
- sans clé, 401 ; avec la bonne clé, 200 ;
- /health reste public et l'API est healthy dans docker compose ps ;
- la page, sur le port 8080, prédit par Nginx ;
- les trois expériences et l'arrêt de l'API sont expliqués ;
- la pull request est fusionnée et le tag tp5 existe.

En cas de problème

Symptôme	Solution
Cannot connect to the Docker daemon	démarrez Docker Desktop et attendez le moteur Linux
COPY models/ : not found	entraînez d'abord : python -m logiai.train
ModuleNotFoundError dans le conteneur	une dépendance manque dans requirements.txt (ou pyproject.toml) : ajoutez-la et reconstruisez
port is already allocated	un autre conteneur ou uvicorn occupe le port : docker ps, ou changez le port côté Windows
conflict : container name already in use	docker ps -a, puis docker rm du conteneur de test
toutes les requêtes : 401, même avec la bonne clé	.env absent de env_file, ou conteneur non recréé après modification du .env
/health indique auth_configured: false	les variables du .env n'arrivent pas au conteneur : env_file
l'API est unhealthy	docker compose logs api ; /health protégé par erreur ; uvicorn sur 127.0.0.1
404 sur /api/v1/predict	API_URL de la page ou location /api/ du fichier Nginx
502 Bad Gateway	api arrêté, nom incorrect, ou uvicorn sans --host 0.0.0.0
ma modification n'apparaît pas	l'image est figée : docker compose up -d --build
la page web n'a pas changé	web/Dockerfile copie la page au build : reconstruisez le service web
.env apparaît dans git status	.gitignore modifié ; déjà poussé : git rm --cached .env, commit, push, et changez les clés

Symptôme	Solution
docker stats montre un autre conteneur	normal : il liste tous les conteneurs actifs

Pour aller plus loin

Si vous avez fini : exécutez l'API avec un utilisateur non-root (instruction USER) ; séparez les dépendances de test et d'exécution pour alléger l'image ; ne publiez plus que le port 8080 et vérifiez que /docs reste joignable autrement ; remplacez les clés de démonstration par des clés longues et aléatoires. Notez ce qui, dans votre configuration, resterait insuffisant pour la production.

Ticket de sortie (individuel, sur papier)

Nom : _____ Groupe : _____

1. Quelle différence entre une image et un conteneur ?
2. Pourquoi le navigateur appelle-t-il /api/v1/predict et non api:8000 ?
3. Pourquoi depends_on seul ne suffit-il pas, et que change service_healthy ?
4. Qu'est-ce qui reste flou pour vous ?